

Лабораторная работа №14

Программирование в командном процессоре ОС UNIX

Липатников Андрей

2026-10-08

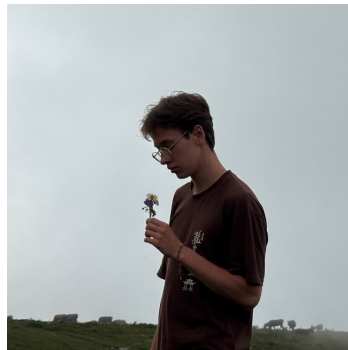
Содержание I

- 1 Информация
- 2 Вводная часть
- 3 Содержание исследования
- 4 Результаты
- 5 Заключение

Раздел 1

Информация

- Липатников Андрей
- Студент НКАбд-04-25
- Российский университет дружбы народов
- 1032253853@rudn.ru



Раздел 2

Вводная часть

- `bash` — основной инструмент автоматизации в UNIX/Linux

- bash — основной инструмент автоматизации в UNIX/Linux
- семафоры нужны для синхронизации процессов

- `bash` — основной инструмент автоматизации в UNIX/Linux
- семафоры нужны для синхронизации процессов
- `man`-страницы — основной источник справки

- `bash` — основной инструмент автоматизации в UNIX/Linux
- семафоры нужны для синхронизации процессов
- `man`-страницы — основной источник справки
- `$RANDOM` — простой способ генерации случайных данных

Объект и предмет исследования

- **Объект:** командная оболочка ОС UNIX
- **Предмет:** семафоры, работа с архивами tar, генерация случайных данных

- **Цель:** изучить основы программирования в оболочке ОС UNIX; научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.
- **Задачи:**
- написать механизм семафоров для синхронизации процессов

- **Цель:** изучить основы программирования в оболочке ОС UNIX; научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.
- **Задачи:**
 - написать механизм семафоров для синхронизации процессов
 - реализовать упрощённый аналог команды `map`

- **Цель:** изучить основы программирования в оболочке ОС UNIX; научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.
- **Задачи:**
 - написать механизм семафоров для синхронизации процессов
 - реализовать упрощённый аналог команды `map`
 - написать генератор случайных букв через `$RANDOM`

- **Цель:** изучить основы программирования в оболочке ОС UNIX; научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.
- **Задачи:**
 - написать механизм семафоров для синхронизации процессов
 - реализовать упрощённый аналог команды `man`
 - написать генератор случайных букв через `$RANDOM`
 - ответить на контрольные вопросы

- ОС: Fedora 41 (Sway), оболочка bash
- инструменты: bash, zcat, groff, less, man, seq
- рабочий каталог: `~/work/study/2025-2026/XXXXXXXXXXXXXXXXXXXX/XXXXXXXX/os-intro/labs/lab14`
- 3 скрипта, 8 снимков

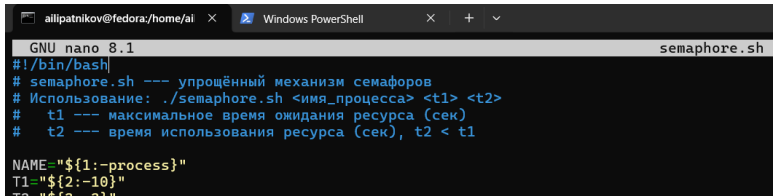
Раздел 3

Содержание исследования

Задание 1. Механизм семафоров

- Синхронизация процессов через **файл-маркер** /tmp/semaphore.lock
- Пока файл существует — ресурс **занят**, другие процессы **ждут**
- После освобождения — следующий процесс **захватывает** ресурс

```
while [ -f "$LOCKFILE" ] && [ "$waited" -lt "$T1" ]
do
    echo "[ $NAME ] ██████████ ████████. █████..."
    sleep 1
    waited=$((waited + 1))
done
```



The screenshot shows a terminal window with two tabs: 'aillipatnikov@fedora:/home/ai' and 'Windows PowerShell'. The active tab is 'aillipatnikov@fedora:/home/ai', which is running the GNU nano 8.1 text editor. The editor is editing a file named 'semaphore.sh'. The content of the script is as follows:

```
#!/bin/bash
# semaphore.sh --- упрощённый механизм семафоров
# Использование: ./semaphore.sh <имя_процесса> <t1> <t2>
#   t1 --- максимальное время ожидания ресурса (сек)
#   t2 --- время использования ресурса (сек), t2 < t1

NAME="${1:-process}"
T1="${2:-10}"
T2="${3:-21}"
```

Задание 1. Запуск

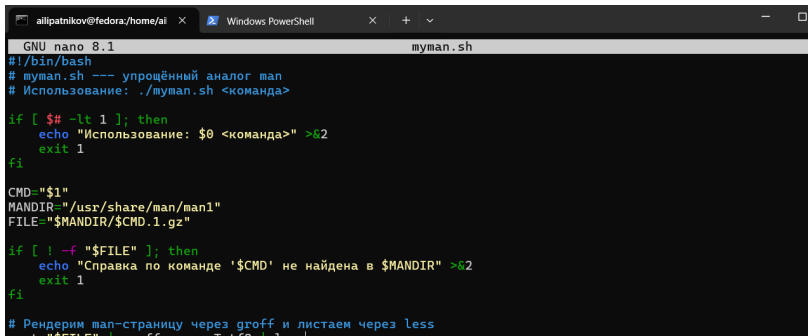
- **Терминал 1:** `./semaphore.sh proc1 15 5` — на переднем плане
- **Терминал 2:** `./semaphore.sh proc2 15 5 > /dev/pts/N &` — в фоне
- Процессы синхронизируются через `lockfile`

```
aillipatnikov_os-intro [Работает] - Oracle VirtualBox
Файл  Машина  Вид  Ввод  Устройства  Справка
+8C  08G38G  FULL  40%  2026-10-10  0002
fedora 0 1 bash
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/
/labs/lab14$ ls
presentation report semaphore.sh
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/
/labs/lab14$ ./semaphore.sh proc1 15 5
[proc1] Старт: ожидаю ресурс (до 15с)
[proc1] Ресурс захвачен. Использую 5с.
^[[1;[proc1] Ресурс освобождён.
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/
/labs/lab14$ tty
/dev/pts/3
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/
/labs/lab14$ [proc2] Старт: ожидаю ресурс (до 15с)
[proc2] Ресурс захвачен. Использую 5с.
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/
/labs/lab14$ ./semaphore.sh proc1 15 5
[proc1] Старт: ожидаю ресурс (до 15с)
[proc1] Ресурс захвачен. Использую 5с.
[proc2] Старт: ожидаю ресурс (до 15с)
[proc2] Ресурс занят. Жду... (0/15с)
[proc1] Ресурс освобождён.
aillipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/
/labs/lab14$ ./semaphore.sh proc2 15 5
[proc2] Старт: ожидаю ресурс (до 15с)
[proc2] Ресурс захвачен. Использую 3с.
[proc2] Ресурс освобождён.
[proc2] Старт: ожидаю ресурс (до 5с)
[proc2] Ресурс захвачен. Использую proc1 5
[proc2] Старт: ожидаю ресурс (до 15с)
[proc2] Ресурс освобождён.
[proc3] Старт: ожидаю ресурс (до 15с)
[proc3] Ресурс захвачен. Использую 5
```

Задание 2. myman.sh — аналог man

- Получает **имя команды** аргументом
- Ищет /usr/share/man/man1/<XXXXXXXXXX>.1.gz
- Распаковывает через zcat, рендерит через groff, листает через less
- Если файла нет — **сообщение об отсутствии**

```
zcat "$FILE" | groff -man -Tutf8 | less
```



```
GNU nano 8.1 myman.sh
#!/bin/bash
# myman.sh --- упрощённый аналог man
# Использование: ./myman.sh <команда>

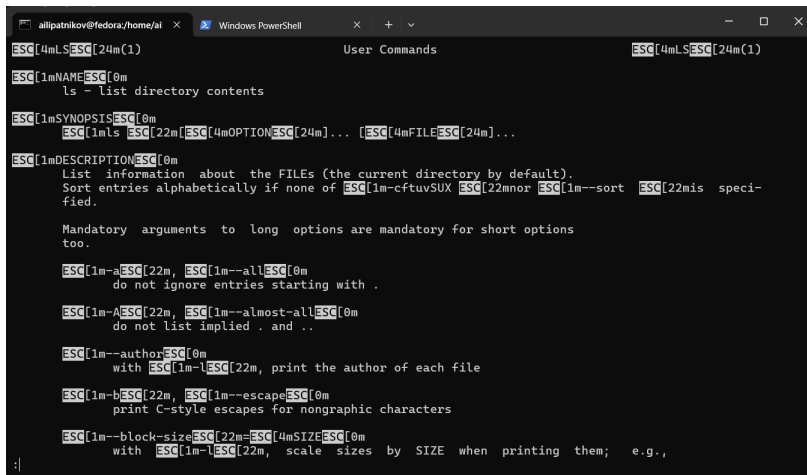
if [ $# -lt 1 ]; then
    echo "Использование: $0 <команда>" >&2
    exit 1
fi

CMD="$1"
MANDIR="/usr/share/man/man1"
FILE="$MANDIR/$CMD.1.gz"

if [ ! -f "$FILE" ]; then
    echo "Справка по команде '$CMD' не найдена в $MANDIR" >&2
    exit 1
fi

# Рендерим man-страницу через groff и листаем через less
zcat "$FILE" | groff -man -Tutf8 | less
```

Задание 2. Запуск для ls



```
ESC[4mLSESC[24m(1)                                User Commands                                ESC[4mLSESC[24m(1)

ESC[1mNAMEESC[0m
ls - list directory contents

ESC[1mSYNOPSISESC[0m
ESC[1mls ESC[22m[ESC[4mOPTIONESC[24m]... [ESC[4mFILEESC[24m]...

ESC[1mDESCRIPTIONESC[0m
List information about the FILES (the current directory by default).
Sort entries alphabetically if none of ESC[1m-cftuvSUX ESC[22mnor ESC[1m--sort ESC[22mis speci-
fied.

Mandatory arguments to long options are mandatory for short options
too.

ESC[1m-aESC[22m, ESC[1m--allESC[0m
do not ignore entries starting with .

ESC[1m-AESC[22m, ESC[1m--almost-allESC[0m
do not list implied . and ..

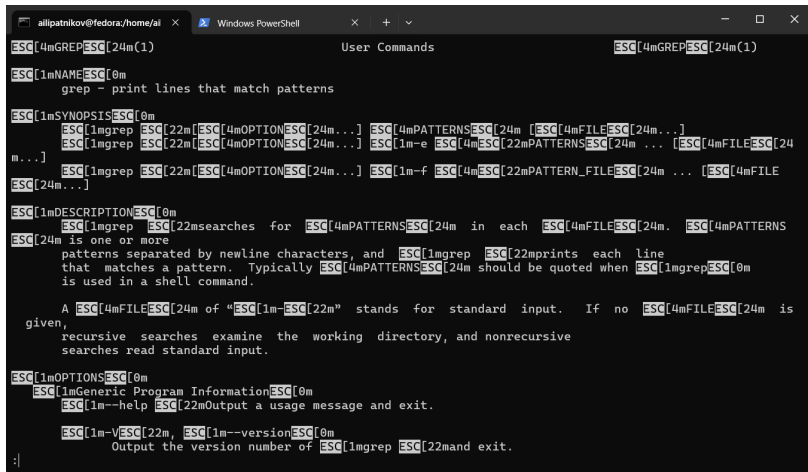
ESC[1m--authorESC[0m
with ESC[1m-lESC[22m, print the author of each file

ESC[1m-bESC[22m, ESC[1m--escapeESC[0m
print C-style escapes for nongraphic characters

ESC[1m--block-sizeESC[22m=ESC[4mSIZEESC[0m
with ESC[1m-lESC[22m, scale sizes by SIZE when printing them; e.g.,
:|
```

Рисунок 4: ./myman.sh ls

Задание 2. Запуск для grep



```
ESC[4mGREPESC[24m(1)                                User Commands                                ESC[4mGREPESC[24m(1)

ESC[1mNAMEESC[0m
  grep - print lines that match patterns

ESC[1mSYNOPSISESC[0m
  ESC[1mgrep ESC[22m[ESC[4mOPTIONESC[24m...] ESC[4mPATTERNSESC[24m [ESC[4mFILEESC[24m...]
  ESC[1mgrep ESC[22m[ESC[4mOPTIONESC[24m...] ESC[1m-e ESC[4mESC[22mPATTERNSESC[24m ... [ESC[4mFILEESC[24m
m...]
  ESC[1mgrep ESC[22m[ESC[4mOPTIONESC[24m...] ESC[1m-f ESC[4mESC[22mPATTERN_FILEESC[24m ... [ESC[4mFILE
ESC[24m...]

ESC[1mDESCRIPTIONESC[0m
  ESC[1mgrep ESC[22msearches for ESC[4mPATTERNSESC[24m in each ESC[4mFILEESC[24m. ESC[4mPATTERNS
ESC[24m is one or more
  patterns separated by newline characters, and ESC[1mgrep ESC[22mprints each line
  that matches a pattern. Typically ESC[4mPATTERNSESC[24m should be quoted when ESC[1mgrepESC[0m
  is used in a shell command.

  A ESC[4mFILEESC[24m of "ESC[1m-ESC[22m" stands for standard input. If no ESC[4mFILEESC[24m is
  given,
  recursive searches examine the working directory, and nonrecursive
  searches read standard input.

ESC[1mOPTIONSESC[0m
  ESC[1mGeneric Program InformationESC[0m
  ESC[1m--help ESC[22mOutput a usage message and exit.

  ESC[1m-VESC[22m, ESC[1m--versionESC[0m
  Output the version number of ESC[1mgrep ESC[22mand exit.

:
```

Рисунок 5: ./myman.sh grep

Задание 2. Сообщение об отсутствии справки

```
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14$ nano myman.sh
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14$ ./myman.sh ls
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14$ ./myman.sh grep
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14$ ./myman.sh nosuchcmd
Справка по команде 'nosuchcmd' не найдена в /usr/share/man/man1
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14$ |
```

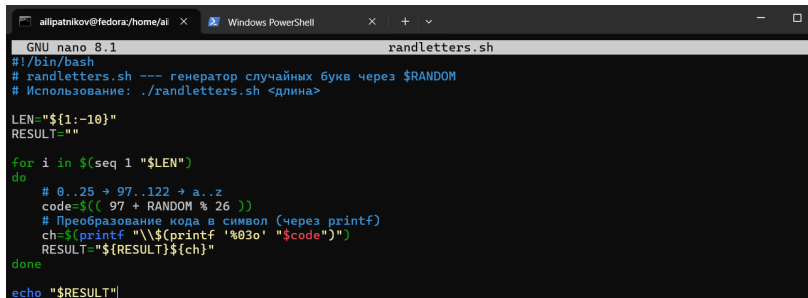
Рисунок 6: ./myman.sh nosuchcmd

- Если файла нет в /usr/share/man/man1 — **сообщение об ошибке**
- Проверка [! -f "\$FILE"]

Задание 3. randletters.sh

- Генерация **случайных букв** a–z через \$RANDOM
- \$RANDOM % 26 → число 0–25
- 97 + . . . → код строчной буквы
- Преобразование кода в символ через printf

```
code=$(( 97 + RANDOM % 26 ))
ch=$(printf "\\$(printf '%03o' "$code")")
```



```
GNU nano 8.1 randletters.sh
#!/bin/bash
# randletters.sh --- генератор случайных букв через $RANDOM
# Использование: ./randletters.sh <длина>

LEN="${1:-10}"
RESULT=""

for i in $(seq 1 "$LEN")
do
    # 0..25 → 97..122 → a..z
    code=$(( 97 + RANDOM % 26 ))
    # Преобразование кода в символ (через printf)
    ch=$(printf "\\$(printf '%03o' "$code")")
    RESULT="${RESULT}${ch}"
done

echo "$RESULT"
```

Задание 3. Запуск

```
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14$ chmod +x randletters.sh
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14$ ./randletters.sh 10
eonasafqhr
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14$ ./randletters.sh 20
iatsuaufkpdktbhvkvj
ailipatnikov@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab14$ |
```

Рисунок 8: Запуск randletters.sh

- Длина строки задаётся **аргументом**
- Каждый запуск — **новая** строка
- Пример: `./randletters.sh 10` → `qwertyasdf`

Раздел 4

Результаты

Таблица 1: Сводка результатов

Задание	Файл	Результат
1	<code>semaphore.sh</code>	Синхронизация 3+ процессов
2	<code>myman.sh</code>	Аналог man с рендерингом
3	<code>randletters.sh</code>	Случайные буквы a-z

Таблица 2: Ключевые конструкции

Элемент	Пример
Семафор	<code>[-f "\$LOCKFILE"] && sleep 1</code>
Распаковка	<code>zcat file.1.gz</code>
Рендеринг	<code>groff -man -Tutf8</code>
Постраничный вывод	<code>less</code>
Случайное число	<code>\$RANDOM % 26</code>
Код → символ	<code>printf "\\\$(printf '%03o' \$code)"</code>

Раздел 5

Заключение

bash позволяет решать системные задачи без привлечения внешних языков: синхронизировать процессы через семафоры, реализовывать аналоги стандартных утилит (man) и генерировать случайные данные через \$RANDOM. Навыки применимы при администрировании и автоматизации UNIX-подобных систем.